Letterboxd Mini-Project — Java Sockets & System Design

From your existing C socket knowledge to a working Java client-server design for a simplified Letterboxd

Contents

How this maps onto your assignment **2**

PART 1 — Java Socket Programming, mapped directly from C **2**

1.1 The core classes, vs. the C functions you already know . . . 2
1.2 A minimal server — direct comparison . . . 3
1.3 A minimal client . . . 3
1.4 Solving the "short read" problem — Java does this for you . . . 3
1.5 Try-with-resources — automatic cleanup, no `close()`-in-every-branch bookkeeping . . . 4
1.6 Concurrency: `fork()` → Threads — and why this is genuinely different, not just a syntax swap . . . 4
1.7 Sending structured data: three real options, pick one and justify it . . . 6

PART 2 — System design for a simplified Letterboxd **6**

2.1 What real Letterboxd actually does (trim this down for your simplified spec) . . . 6
2.2 Suggested simplified feature set for your implementation . . . 7
2.3 Architecture — three tiers, sockets in the middle . . . 7
2.4 Message protocol — define this explicitly, it's your actual "distributed systems" artifact here . . . 7
2.5 Suggested class/package structure . . . 8
2.6 Where the "distributed computing" angle genuinely shows up (for your other two documents) . . . 8

PART 3 — Resources **8**

Java socket programming . . . 8
Concurrency in Java . . . 9
JSON / serialization . . . 9
Embedded persistence . . . 9
Distributed systems design reading (for `ArchitectureOfApp` and `ParadigmsandAlgorithms`) . . . 9
Letterboxd-specific . . . 9
Diagramming / documentation tools . . . 9

How this maps onto your assignment

Steps 1–4 of your assignment (app study, paradigms/algorithms, requirement spec) are separate `.doc` deliverables you'll write once you've actually built something and understand it — this document is deliberately scoped to **Step 5, the implementation**, since that's what you're tackling first. Everything here is meant to feed back into those other documents once you write them: the design in Part 2 is what your `RequirementSpecificationSimpleApp` will formalize, and the client-server/threading choices in Part 2 are exactly what your `ParadigmsandAlgorithms` document should later name explicitly (client-server paradigm, concurrency model, replication/consistency choices if you add them).

Part 1 takes every C socket function you already know and gives you its direct Java equivalent — you already understand *what* each step does conceptually (from your TCP lab), so this is almost entirely a translation exercise, plus a few things Java genuinely does better than raw C sockets. **Part 2** is the actual system design for a simplified Letterboxd, sockets included but not the whole story. **Part 3** is a curated resource list.

PART 1 — Java Socket Programming, mapped directly from C

1.1 The core classes, vs. the C functions you already know

C (what you know)	Java equivalent	Notes
`socket()` + `bind()` + `listen()`	`new ServerSocket(port)`	One constructor call does all three — Java hides the socket/bind/listen split entirely.
`accept()`	`serverSocket.accept()`	**Blocks**, exactly like C's `accept()`. Returns a `Socket` object (like C's returned `connfd`) — the original `ServerSocket` keeps listening.
`socket()` + `connect()`	`new Socket(ip, port)`	One constructor does both — throws `IOException` if the connection fails, instead of returning `-1`.
`read()` / `write()` on the fd	methods on `socket.getInputStream()` / `socket.getOutputStream()`	You get an `InputStream`/`OutputStream` object from the socket, then read/write through *that*, not the socket object directly.
`close()`	`socket.close()`	Same idea; Java also supports **try-with-resources** (§1.5) to close automatically.
checking return value `< 0`, `perror()`	**exceptions** (`IOException`, `UnknownHostException`, etc.)	Java doesn't use sentinel return values for errors — a failed operation **throws**, and you `catch` it. This is the single biggest structural difference from C's style.

1.2 A minimal server — direct comparison

C (what you already wrote):

```c
listenfd = socket(AF_INET, SOCK_STREAM, 0);
bind(listenfd, (struct sockaddr*)&servaddr, sizeof(servaddr));
listen(listenfd, LISTENQ);
connfd = accept(listenfd, NULL, NULL);
read(connfd, buf, sizeof(buf));
write(connfd, buf, n);
close(connfd);
```

Java equivalent:

```java
import java.io.*;
import java.net.*;

ServerSocket serverSocket = new ServerSocket(port);      // socket()+bind()+listen()
Socket clientSocket = serverSocket.accept();               // accept() -- blocks

InputStream in   = clientSocket.getInputStream();
OutputStream out = clientSocket.getOutputStream();

byte[] buf = new byte[1024];
int n = in.read(buf);          // like read()
out.write(buf, 0, n);          // like write()

clientSocket.close();
```

Notice there's no `sockaddr_in`, no `htons`/`htonl`, no `memset` — Java's `ServerSocket`/`Socket` constructors handle all of that internally. **You do not need `htons`/`htonl` in Java at all** — the classes handle network byte order for you; this is one of the things Java genuinely simplifies versus raw C sockets.

1.3 A minimal client

```java
Socket socket = new Socket("127.0.0.1", port);   // socket()+connect()
OutputStream out = socket.getOutputStream();
InputStream in    = socket.getInputStream();

out.write("hello".getBytes());
byte[] buf = new byte[1024];
int n = in.read(buf);
System.out.println(new String(buf, 0, n));

socket.close();
```

1.4 Solving the "short read" problem — Java does this for you

Remember Part 5 of your TCP lab guide — a raw `read()` on a TCP socket in C isn't guaranteed to return all the bytes you asked for, which is why you wrote `readn()`/`writen()` wrapper functions. **Java has this solved for you, built in**, if you use `DataInputStream`/`DataOutputStream`:

```java
import java.io.*;

DataInputStream  dataIn  = new DataInputStream(socket.getInputStream());
DataOutputStream dataOut = new DataOutputStream(socket.getOutputStream());
```

```java
// Sending a fixed-size, typed message -- like your C `struct mesg`:
dataOut.writeInt(seqNo);            // 4 bytes, network byte order, ALWAYS
dataOut.writeShort(reserved);       // 2 bytes -- keep this 0
dataOut.write(dataBytes, 0, 16);    // fixed 16 raw bytes
dataOut.flush();

// Receiving it back, on the other end:
int seqNo    = dataIn.readInt();       // blocks/loops internally until 4 full bytes arrive
short reserved = dataIn.readShort();
byte[] data = new byte[16];
dataIn.readFully(data);                 // THIS is your readn() -- guaranteed to read exactly 16 bytes or throw
```

`readFully()` is the direct equivalent of the `readn()` function you wrote by hand in C — it loops internally until it has read exactly the number of bytes you asked for, or throws an `EOFException` if the stream ends first. `writeInt()`/`writeShort()`/`write()` on a `DataOutputStream` are similarly guaranteed to either fully succeed or throw — you don't need to hand-write a `writen()` loop in Java. This is worth stating explicitly in your `ArchitectureOfApp`/paradigms document later, since it's a genuine, nameable design difference between the two languages' networking APIs.

1.5 Try-with-resources — automatic cleanup, no `close()`-in-every-branch bookkeeping

```java
try (ServerSocket serverSocket = new ServerSocket(port)) {
    while (true) {
        Socket client = serverSocket.accept();
        // handle client...
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Anything declared inside the `try (...)` parentheses that implements `AutoCloseable` (which `Socket`/`ServerSocket`/streams all do) is **automatically closed** when the block exits — normally or via an exception — without you writing explicit `close()` calls in every possible exit path. In C you had to be careful to `close()` on every branch, including error paths; Java's compiler enforces this for you structurally.

1.6 Concurrency: `fork()` → Threads — and why this is genuinely different, not just a syntax swap

In C, your concurrent server `fork()`ed a whole new **process** per client — separate address space, separate PID, needed a `SIGCHLD` handler to avoid zombies.

In Java, the idiomatic equivalent is a new **thread** per client — same address space (all threads share the JVM's heap), much lighter weight than a process, and **there is no zombie-process problem in Java at all**, since threads aren't processes and don't need reaping the way child processes do.

```java
import java.io.*;
import java.net.*;

public class LetterboxdServer {
    public static void main(String[] args) throws IOException {
        int port = Integer.parseInt(args[0]);
        ServerSocket serverSocket = new ServerSocket(port);
        System.out.println("Server listening on port " + port);
```

```
        while (true) {
            Socket clientSocket = serverSocket.accept();     // blocks, like accept()
            new ClientHandler(clientSocket).start();          // hand off to a new thread, keep accepting
        }
    }
}

class ClientHandler extends Thread {
    private Socket socket;
    ClientHandler(Socket socket) { this.socket = socket; }

    public void run() {
        try {
            DataInputStream  in  = new DataInputStream(socket.getInputStream());
            DataOutputStream out = new DataOutputStream(socket.getOutputStream());
            // ... handle this one client's requests, loop until they disconnect ...
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            try { socket.close(); } catch (IOException e) { /* ignore */ }
        }
    }
}
```

This `Thread`-per-client pattern is the direct structural equivalent of your `fork()`-per-client C server — same design shape (accept in a loop, hand off, go back to accepting), different unit of concurrency underneath. Since your shared data (users, diary entries, films) now lives in **one process's heap**, shared **by all threads automatically** — you get the "shared memory" your distributed computing labs made you build manually in C (via `shm_open`/`mmap`) completely for free, but that also means you must now protect shared data structures with **Java's `synchronized` keyword or a `java.util.concurrent` lock**, exactly the same mutual-exclusion problem as your mutex labs, just phrased in Java's syntax:

```
class UserDatabase {
    private final Map<String, User> users = new HashMap<>();

    public synchronized void addUser(User u) {     // like pthread_mutex_lock()/unlock() around this method
        users.put(u.getUsername(), u);
    }

    public synchronized User getUser(String username) {
        return users.get(username);
    }
}
```

`synchronized` on a method is equivalent to wrapping the entire method body in `pthread_mutex_lock()`/`pthread_mutex_u` on an implicit, per-object lock — same mutual-exclusion guarantee, much less boilerplate than raw pthreads.

For a class project, a thread pool (`ExecutorService`) is a nicer alternative to raw `new Thread()` per client, since it caps how many threads run concurrently instead of spawning unbounded threads under heavy load:

```
import java.util.concurrent.*;

ExecutorService pool = Executors.newFixedThreadPool(20);
while (true) {
```

```
    Socket clientSocket = serverSocket.accept();
    pool.submit(new ClientHandler(clientSocket));   // ClientHandler implements Runnable instead
}
```

1.7 Sending structured data: three real options, pick one and justify it

1. **`DataOutputStream/DataInputStream` with explicit typed fields** (shown in §1.4) — closest to your C `struct mesg` approach; you define the exact wire format field-by-field. Good if you want full control and a fixed, simple protocol.
2. **Java's built-in object serialization** (`Serializable` interface + `ObjectOutputStream/ObjectInputStream`) — let Java automatically serialize/deserialize whole objects:

   ```
   class LogEntry implements Serializable {
       String username, filmTitle, review;
       int rating;
   }
   // sending:
   ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream());
   oos.writeObject(new LogEntry(...));
   // receiving:
   ObjectInputStream ois = new ObjectInputStream(socket.getInputStream());
   LogEntry entry = (LogEntry) ois.readObject();
   ```

   Simplest to write, but the wire format is Java-specific (not portable to a non-Java client) and slightly heavier than a hand-designed format.
3. **A text-based protocol (plain strings or JSON)** over `BufferedReader/PrintWriter`, optionally with a library like **Gson** for JSON serialization — most flexible, human-readable (great for debugging/screenshots in your submission), and the standard choice for most real client-server protocols (including, loosely, how Letterboxd's own app likely talks to its backend via HTTP/JSON):

   ```
   // Sending: {"type":"LOG_FILM","username":"srinidh","film":"Amelie","rating":5}
   PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
   out.println(jsonString);

   BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
   String line = in.readLine();
   ```

Recommendation for this project: option 3 (JSON over a line-based text protocol) — it's the easiest to demo in screenshots, easiest to debug by eye, and gives you a natural, well-defined **message protocol** to document in your `RequirementSpecificationSimpleApp` (Part 2.4 below sketches exactly this).

PART 2 — System design for a simplified Letterboxd

2.1 What real Letterboxd actually does (trim this down for your simplified spec)

Core features: users log films they've watched (with an optional rating 0.5–5 stars, review text, rewatch flag, watch date); a personal **diary** (chronological log); **watchlist** (films to watch later); custom **lists**; following other users and seeing their activity in a **feed**; a searchable film **catalog** with aggregate ratings. (Confirmed from public sources: Letterboxd's actual backend is a Java-based in-house CMS, PostgreSQL with read replicas, Redis for caching, Varnish for page caching,

nginx/Tomcat serving, and in-house load balancing — worth citing in your `ArchitectureOfApp` document, see Part 3's resources.)

2.2 Suggested simplified feature set for your implementation

Pick a subset that's demoable in the time you have — a reasonable "core loop" for a class project: - **Register / Login** (username + password, no need for real security like hashing to start — but add basic hashing if you have time, worth a sentence in your paradigms doc about why real systems never store plaintext passwords). - **A small, fixed film catalog** (hardcode 30–50 real films with title/year/director — no need to integrate a real movie database API unless you want to). - **Log a film**: rating (1–5), optional review text, date — stored in the user's diary. - **View your own diary.** - **View another user's public diary** (by username). - **Follow / unfollow another user.** - **A simple feed**: recent diary entries from users you follow. - **Search the catalog** by title substring.

Deliberately cut for scope, but worth naming as "future work" in your requirement spec: lists, watchlist, comments/likes on reviews, a real recommendation algorithm, a web frontend (a console client is entirely fine and expected for a sockets project).

2.3 Architecture — three tiers, sockets in the middle

```
 [ Java Console Client(s) ]  <--- TCP sockets, JSON messages --->  [ Java Multithreaded Server ]  <--->  [ Persistence Layer ]
```

- **Client:** a Java console app (or simple Swing/JavaFX GUI if you want to go further) — connects via `Socket`, sends JSON requests, prints responses. One client instance per user session, exactly like your C TCP client, run multiple times to simulate multiple users.
- **Server:** `ServerSocket` accepting connections, one thread (or thread-pool task) per connected client, exactly as in §1.6. Holds the **in-memory** data structures for users/films/diary entries, guarded by `synchronized` methods or explicit locks, **plus** a persistence layer so data survives a server restart.
- **Persistence layer — pick one, simplest first:**
  1. **Simplest:** serialize your in-memory data structures to a JSON file on disk periodically / on shutdown, load it back on startup (use Gson to serialize a top-level `Database` object directly to/from a `.json` file).
  2. **More "real":** an embedded database via JDBC — **SQLite** (via the `sqlite-jdbc` driver, zero server setup needed) is the natural choice for a class project; gives you real SQL, a `users` table, `films` table, `diary_entries` table with a foreign key to both. Either is defensible; SQLite is more impressive for the "distributed systems understanding" angle since it's a closer analogue to how Letterboxd actually uses PostgreSQL, and gives you something concrete to compare against in your `ParadigmsandAlgorithms` document (e.g., discussing ACID transactions, why a real system uses read replicas, etc., even though your version is a single embedded DB).

2.4 Message protocol — define this explicitly, it's your actual "distributed systems" artifact here

A simple newline-delimited JSON protocol. Every client request has a `type` field; every server response has a `status` field.

Example requests (client → server):

```
{"type":"REGISTER", "username":"srinidh", "password":"..."}
{"type":"LOGIN", "username":"srinidh", "password":"..."}
{"type":"LOG_FILM", "username":"srinidh", "filmId":12, "rating":4.5, "review":"Loved it"}
{"type":"GET_DIARY", "username":"srinidh"}
{"type":"FOLLOW", "follower":"srinidh", "followee":"alex"}
```

```
{"type":"GET_FEED", "username":"srinidh"}
{"type":"SEARCH_FILM", "query":"amelie"}
```

Example responses (server → client):

```
{"status":"OK", "message":"Registered successfully"}
{"status":"ERROR", "message":"Username already exists"}
{"status":"OK", "diary":[{"filmId":12,"title":"Amélie","rating":4.5,"review":"...","date":"2025-09-20"}]}
```

Document this table fully in your `RequirementSpecificationSimpleApp_RegNo.doc` — it's effectively your API specification, the same role an actual REST API spec or protobuf schema would play in a real distributed system, just simplified to raw sockets.

2.5 Suggested class/package structure

```
letterboxd/
  server/
    LetterboxdServer.java      // main(), accept loop
    ClientHandler.java          // Runnable, one per connected client
    RequestRouter.java          // parses the JSON "type" field, dispatches to a handler method
  client/
    LetterboxdClient.java       // main(), console menu, sends requests
  model/
    User.java
    Film.java
    DiaryEntry.java
  data/
    Database.java               // the shared, synchronized in-memory store (+ persistence load/save)
  protocol/
    Message.java                // shared request/response helper (build/parse JSON)
```

2.6 Where the "distributed computing" angle genuinely shows up (for your other two documents)

Even a simplified, single-server project touches real distributed-systems ideas worth naming explicitly later: - **Client-server paradigm** itself, obviously — the foundational one. - **Concurrency control** — your `synchronized` methods are a direct, nameable instance of mutual exclusion, the same core idea as distributed locking, just local instead of across machines. - **Optional stretch, genuinely worth attempting for extra depth:** run **two server instances** sharing the same SQLite file (or better, in-memory replicated via a simple periodic sync), with the client picking one via a naive round-robin — this gives you a real, small-scale example of **load balancing** and a taste of the **replication/consistency** questions real distributed systems face (what happens if both instances get a conflicting write at nearly the same time?), directly relevant to whatever paradigms Letterboxd's real read-replica PostgreSQL setup uses. Not required, but a strong way to make your `ParadigmsandAlgorithms` document connect your own implementation back to the real app's architecture, rather than just describing Letterboxd's system in the abstract.

PART 3 — Resources

Java socket programming

- Oracle's official tutorial: *"All About Sockets"* — `docs.oracle.com/javase/tutorial/networking/sockets/`
- `java.net.Socket` / `java.net.ServerSocket` Javadocs — the authoritative reference for every method signature.

- *Java Network Programming*, 4th ed., Elliotte Rusty Harold (O'Reilly) — the closest Java equivalent to Stevens' UNP in depth and rigor.
- Baeldung — "A Guide To Java Sockets" (practical, example-driven, good for quick syntax lookups while coding).

Concurrency in Java

- `java.util.concurrent` package Javadocs (`ExecutorService`, `ConcurrentHashMap`, locks).
- *Java Concurrency in Practice*, Brian Goetz — the standard reference on Java threading correctness, if you want to go deeper than `synchronized`.

JSON / serialization

- Gson (Google) — `github.com/google/gson` — simplest JSON library to drop into a small project.
- Jackson — heavier but more feature-rich, if Gson feels too limited later.

Embedded persistence

- `sqlite-jdbc` (Xerial) — `github.com/xerial/sqlite-jdbc` — zero-setup embedded SQL database, ideal for this project's scale.

Distributed systems design reading (for `ArchitectureOfApp` and `ParadigmsandAlgorithms`)

- *Designing Data-Intensive Applications*, Martin Kleppmann — the best single book connecting real systems (replication, partitioning, consistency) to the concepts your course names.
- Mikito Takada, *"Distributed Systems for Fun and Profit"* (free online book) — short, readable, covers CAP theorem, replication, and consensus at a level that maps well to a course project.
- "An Illustrated Proof of the CAP Theorem" (widely circulated blog post) — good, quick, visual explanation if your paradigms document needs to reference CAP.

Letterboxd-specific

- Letterboxd's own answer on their tech stack (Quora, from the Letterboxd team): confirms a Java-based in-house CMS, Apache Tomcat + nginx, in-house load balancing, PostgreSQL with replicas, Hibernate, Redis for metadata caching, Varnish for page caching — a strong, citable primary-ish source for your `ArchitectureOfApp` document.
- StackShare's Letterboxd page (`stackshare.io/letterboxd/letterboxd`) — a secondary, community-maintained tech-stack summary, useful for corroboration.
- Letterboxd's own blog/help pages, if they publish anything about how features like diary/lists/feed work conceptually (check `letterboxd.com/about` and their blog for any engineering-adjacent posts before citing).

Diagramming / documentation tools

- draw.io (`app.diagrams.net`) — free, good for architecture and sequence diagrams for your requirement spec.
- PlantUML — text-based UML if you'd rather script your diagrams than draw them by hand.

Suggested order of work: get the minimal Java client/server from Part 1 actually connecting and echoing messages first (a tiny, boring win, but it proves your environment/Makefile-equivalent/build setup works) — then layer in the JSON protocol from Part 2.4, then the in-memory `Database` class, then persistence last. Don't attempt the two-server load-balancing stretch goal until the single-server version fully works end to end.